ClaimCheck: Project Proposal

Oct 23, 2026 · @Krish Totala

Summary

ClaimCheck is a campus lost and found app that automatically matches lost item reports with found item posts, even when one side has a photo and the other only has a written description. Before anyone can claim an item, they have to describe a hidden detail that only the real owner would know, which stops people from claiming things that aren't theirs. Matches are also ranked by where and when the item was lost and found, so the most likely candidates show up first. The name comes from the claim check tickets that old railway lost property offices handed out, which ties the project to this year's Steel and Steam theme.

The problem

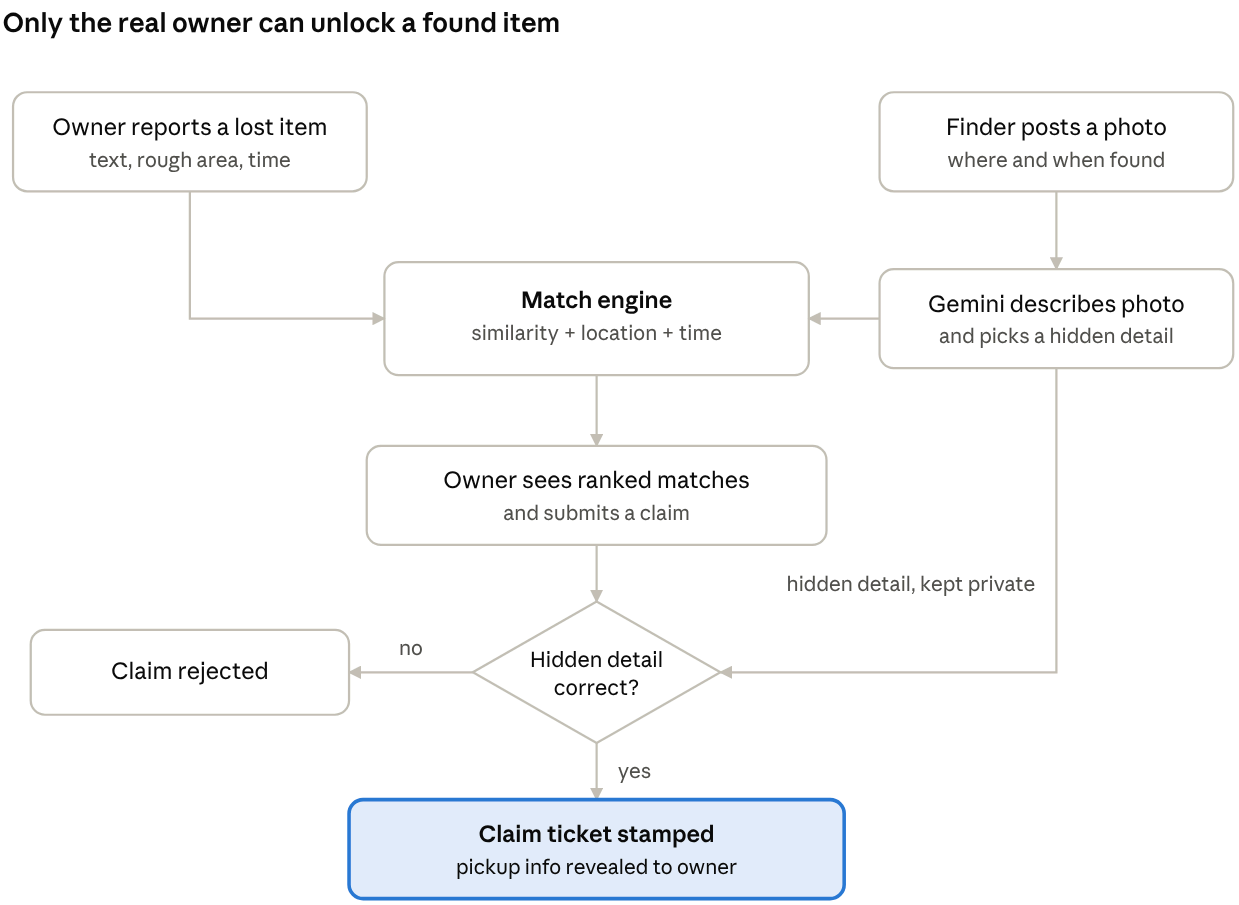
Lost and found on a college campus is scattered across building front desks, dorm offices, Discord servers, Instagram stories, and group chats. If you lose something, you have to check all of them, and you usually give up after one or two. If you find something, you either drop it at the nearest desk or post it in one channel and hope the owner happens to see it.

Existing apps mostly work like a bulletin board where you scroll through a list of items. That breaks down in three ways:

- **The two sides describe things differently.** The finder has the item in hand and can take a photo. The owner usually has no photo and writes something vague like "black water bottle with stickers." Keyword search can't connect those.
- **Anyone can claim anything.** If a pair of AirPods is posted publicly, nothing stops the wrong person from saying they're theirs.
- **People don't know exactly where they lost it.** They know roughly where they were and when, but bulletin boards don't use that information at all.

ClaimCheck is built around fixing those three things. Before we pitch, we should ask 15 to 20 people at the hackathon whether they've lost something on campus and whether they got it back, so we can open with real numbers.

How it works



ClaimCheck flow · lost and found sides meet in the match engine

The owner and the finder each post from their own side, and the match engine connects them using the description, location, and time. The hidden detail never appears publicly and is only used to check a claim, so a match alone isn't enough to get the item.

Core features

1. Matching text to photos

When a finder uploads a photo, Gemini's vision model writes a detailed description of the item (type, color, brand, stickers, damage, anything written on it). We then turn every description, from both lost and found posts, into an embedding so similar items land close together. A vague lost report like "black Hydro Flask with a Yosemite sticker" can then be

compared directly against a found photo of that bottle. If time allows, we add CLIP image embeddings so photo-to-photo matches work too, for cases where the owner does have a picture.

2. Ownership verification

When a finder uploads a photo, the AI picks one or two distinguishing details that would only be known by the owner, like a scratch on the case, a name written on the tag, or the color of a phone case. Those details are hidden from the public listing. Anyone who tries to claim the item has to describe that detail in their own words, and Gemini checks whether their answer matches. The finder can also add their own private detail if they want. Only a passed check reveals the finder's contact info or pickup location.

3. Location and time awareness

People rarely know exactly where they lost something, so the lost form asks for a rough area on a campus map and a time window instead of a single point. Found posts record where and when the item was picked up. A found item near the lost area and inside the time window gets a higher score than an equally similar item found across campus a week earlier.

4. Proactive alerts

You don't have to keep checking the app. When a new found item scores above a threshold against your open lost report, you get notified. This works in both directions, so a finder can see that someone already reported their item as lost.

5. Solving the empty-app problem

A matching app only works if both sides use it. We plan for a bulk upload mode where a front desk or dorm office can photograph everything in their bin at once, and each photo becomes its own found post automatically. For the demo, we can show this with a batch of photos, and in the pitch we explain that partnering with desks is how the app gets enough items on day one.

Theme fit: Steel and Steam

The app is styled as an old railway lost property office. Lost and found posts appear as luggage tags, a successful claim is stamped like a ticket, and the matching screen can look like a station departures board listing likely matches. This is mostly design work, but it

shows the judges we took the theme seriously and gives us a real shot at the UI/UX prize. We should keep it readable and not let the styling get in the way of the matching, which is the actual product.

Tech stack

This is a suggested stack chosen for speed with Claude Code. We can swap anything the team is more comfortable with.

Part	Suggested tool	What it does
Frontend	Next.js + Tailwind	Lost and found forms, match feed, claim flow, themed UI
Campus map	Leaflet with a campus map image or OpenStreetMap	Picking a rough lost area and a found location
Database and auth	Supabase (Postgres + storage + auth)	Users, posts, photos, claims
Vector search	pgvector inside Supabase	Storing embeddings and finding nearest matches
Photo descriptions	Gemini vision	Turning each found photo into a detailed text description and picking hidden details
Embeddings	Gemini text embeddings (CLIP as a stretch)	Putting lost and found descriptions in the same space
Verification judge	Gemini	Checking whether a claimant's answer matches the hidden detail
Hosting	Vercel	Live URL for judges and the Devpost link

Match score. Each lost and found pair gets one score that combines description similarity, whether the category matches, distance between the lost area and the found location, and whether the found time falls inside the lost window. We start with simple hand-tuned weights and adjust them against our seeded test items.

Scope

We build in this order and don't start the next tier until the one above it works end to end.

Must have (the demo depends on these)

- Post a lost item with a description, category, rough area, and time window
- Post a found item with a photo, location, and time
- Gemini description of found photos and embedding-based matching
- Ranked match list for a lost report
- Hidden detail generation and the claim verification check
- About 30 realistic seeded items so the demo has something to match against

Should have

- Location and time weighting in the match score
- Campus map for picking areas
- The railway lost property styling
- In-app match alerts

Stretch

- Bulk upload for front desks
- CLIP image-to-image matching
- Email or SMS alerts
- Automatically blurring or rejecting photos of IDs and cards

Timeline

Devpost submissions are due Sunday at 9:00 AM, and judging runs 10:30 AM to 1:00 PM in SAC Ballroom A.

1. **Friday 9 PM to 2 AM:** Set up the repo, Supabase, and Vercel. Get one found photo through Gemini and one lost description matched against it. This is the riskiest part, so it goes first.
2. **Saturday morning:** Lost and found forms, photo upload, and the ranked match list working end to end.
3. **Saturday afternoon:** Hidden detail generation and the claim verification flow. Start writing the 30 seeded items.

4. **Saturday evening:** Location and time weighting, campus map, and the railway styling. Run the student survey during dinner.
5. **Saturday night:** Alerts and any stretch goals, otherwise bug fixing. Feature freeze by about 4 AM.
6. **Sunday 4 AM to 9 AM:** Record a backup demo video, write the Devpost page, rehearse the pitch, and sleep in shifts if possible.

Team roles

This split assumes four people. With three, the AI and backend roles merge. Everyone runs their own Claude Code session on a separate part of the repo so we don't step on each other.

Role	Owns	Name
AI and matching	Gemini descriptions, embeddings, match score, verification judge	
Backend	Supabase schema, pgvector, API routes, alerts	
Frontend and design	Forms, match feed, claim flow, map, railway styling	
Product and pitch	Seeded data, student survey, Devpost page, demo script, testing	

Demo plan

The live demo should take about a minute and tell the whole story.

1. Open with the survey result, for example how many of the students we asked had lost something on campus and never got it back.
2. A teammate posts a found item by taking a photo of a real object at the table, like someone's water bottle or AirPods case.
3. Another teammate reports it as lost with only a vague text description and a rough area on the map. The found item appears at the top of their matches.
4. A third person tries to claim it without knowing the hidden detail and gets rejected.
5. The real owner answers the verification question correctly and the claim ticket gets stamped.

6. Close with the bulk upload idea and how front desks would get the app populated on day one.

We should test the exact demo objects ahead of time and keep a recorded backup video in case the Wi-Fi or an API fails during judging.

Prizes we can target

Category	Prize	Why we fit
Best Overall (1st to 3rd)	$1,000 cash, Apple Watch SE 3, Bambu Lab A1 Mini	Real problem, novel matching and verification, polished demo
Best UI/UX	Insignia 24″ Smart Fire TV	Railway lost property styling and a clean claim flow
MLH Best Use of Gemini API	Swag kit	Gemini handles descriptions, hidden details, and verification

The MLH category list should be confirmed on the event Devpost or at the MLH table, since the public MLH page may be out of date. Prize details are from the SBUHacks 2026 site and the MLH prizes page.

Risks and open questions

Risk	What we do about it
Matching returns the wrong items	Test early with the seeded set and tune the score weights; show the top 3 matches instead of only one
Verification is too strict or too loose	Have Gemini return a confidence level, and let borderline answers go to the finder to approve
Hidden detail is too obvious from the photo	Crop or blur that region in the public photo, or only show a generic description publicly
Photos contain IDs, cards, or personal info	Never show IDs publicly; flag them as sensitive and route them to a front desk instead

Risk	What we do about it
API rate limits or Wi-Fi drop during judging	Cache Gemini results for seeded items and keep the backup video ready
Judges say lost and found apps already exist	Lead the pitch with verification and cross-format matching, which those apps don't have

Open questions for the team

- [ ] Are we all comfortable with the suggested stack, or do we swap anything?
- [ ] Who takes which role?
- [ ] Do we want a mobile-first layout, since most people would post from their phone?
- [ ] Who runs the student survey and when?